MuST-Walk: Multi-Scale Temporal Walk Encodings for Efficient Temporal Link Prediction

DAL 518: Graph Machine Learning, Project 4.3

Abhi Raj (23125003) Manas Dnyaneshvar Patil (23125020)
Kamranjeet Singh (23125016)

November 2026

Code and results: https://github.com/MDP2004/MuST-Walk

Abstract

Temporal link prediction asks which pair of nodes will interact next in a stream of timed edges. TPNet encodes temporal walks efficiently with a random-projection sketch, but it gives each dataset one exponential decay rate, and so one memory length. We ask whether several rates on one shared projection do better, at what cost, and on which graphs. We propose two methods: MuST-Walk keeps J sketches over one projection and fuses them, and MuST-HN adds training on historical negatives. We prove the walk recursion and the sketch identity from first principles, extend the projection guarantee to J scales, derive the exact noise of each pair feature, and give a lazy update that removes the batch-wide write from the per-edge cost. Inspecting the pipeline, we remove a same-timestamp leak and show that on yearly data the sketch reads a one-step-old edge at full weight. We evaluate on six DyGLib and four TGB datasets against TPNet. MuST-Walk passes our per-dataset rule on 3 of 6 DyGLib datasets but wins 9 of 18 seeds, and its cross-dataset test is not supported. MuST-HN beats TPNet by $+9.01$ to $+26.48$ AP on all six datasets, 18 of 18 seeds, though that gain comes from the training change and not from multi-scale, and it costs random-negative AP on 4 of 6 datasets. To explain these results we measure each graph's structure and test each decay rate with no neural model. Most of the multi-scale gain has one cause. A pair never seen in its stored direction cannot be a historical negative, so it must be real. Fast decay reads such a pair and a long-idle pair as the same zero, and slower scales separate them. With those pairs removed, the feature-level ladder gain falls from $+13.65$ to $+1.32$ AP on US Legis. and from $+27.77$ to $+5.18$ on Can. Parl. A no-learning rule that uses this fact beats both proposed methods on four of six datasets. On near-complete graphs walks saturate, and the sketch is blind to edge direction, which explains the failures there.

1 Problem statement

A continuous-time dynamic graph is a stream of timestamped edges. Given all edges before time t, temporal link prediction scores whether nodes u and v interact at t. The task underlies recommendation, communication analysis and forecasting of relations between institutions. Two signals dominate it: pairs that interacted recently tend to interact again, and pairs connected through shared neighbours tend to interact next.

This project studies scalable, time-aware structural encodings based on temporal walks rather than static paths. Walk encodings can capture how nodes are connected through time, but counting walks exactly costs too much on a stream. The goal is to keep the efficiency of a sketch while capturing reachability over long time ranges, and to understand when doing so helps.

1.1 Formulation

Let $\mathcal{G} = \{(u_i, v_i, t_i)\}_{i=1}^{E}$ with $t_1 \le t_2 \le \ldots$, on n nodes. Pairs are stored in an order (u, v), and we call that the stored direction. The model outputs a score for a query (u, v, t) using only edges strictly before t. Data is split in time order, 70/15/15. We report average precision,

$$\mathrm{AP} = \sum_i (R_i - R_{i-1})\, P_i, \tag{1}$$

over distinct score thresholds with recall R_i and precision P_i, so tied scores form one step. Positives are real test edges. A test needs negatives, and the choice defines the task.

Definition 1 (Negative types)**.** A *random* negative pairs a query source with a random destination. A *historical* negative [3] is a stored pair (u, v) seen strictly before the batch and not active in the batch's time interval. An *inductive* negative involves unseen nodes. We use historical AP as the primary metric.

1.2 Research questions and findings

- **RQ1. Do several decay rates on one projection beat one?** Not in general. MuST-Walk passes a per-dataset rule on 3 of 6 datasets, and it wins 9 of 18 seeds (Section 6).
- **RQ2. On which graphs, and why?** Where fast decay ties never-seen pairs with long-idle pairs, on graphs with turnover and many new pairs. Most of that gain comes from how the historical test is built (Section 7).
- **RQ3. What does it cost?** Time and memory linear in J, with no new approximation in the sketch, and a lazy update that removes the batch-wide write (Section 4.7).
- **RQ4. What do these benchmarks measure?** Random AP is capped near a computable level on near-complete graphs, and historical AP gives a share of positives away (Section 7.1).

Our contributions are two methods, MuST-Walk and MuST-HN, a self-contained treatment of the walk sketch with exact noise formulas, the lazy update, a corrected pipeline, and a structural explanation of each outcome that we test directly. We compare each method with the TPNet baseline only; other configurations we ran are outside the scope of this paper. We do not claim historical-negative training or the leak fix as new; both have prior work, cited where used.

2 Literature review

2.1 Approaches to temporal link prediction

Work on temporal link prediction falls into a few families. Memory-based models such as TGN [5] keep a state per node, updated by events. Walk-sampling models such as CAWN [6] sample anonymous walks around each pair. Sequence models such as DyGFormer [4] encode co-occurrence in first-hop histories. Encodings built on a whole supra-graph, such as SLATE [7], apply a Laplacian encoding to a multi-layer graph. TPNet [1] shows that many relative encodings are choices of a walk score and a readout, and it maintains the walk matrix through a random projection. TPNet is our baseline, and the next two subsections review it.

Three further lines of work shape this project. On evaluation, Poursafaei et al. [3] introduced historical and inductive negatives, and TGB [2] provides standard benchmarks. On training, ENS [11], CurNM [12] and ATNSF [13] use harder negatives. On batching, Zhang et al. [9] and Lampert et al. [10] study how same-time information reaches batched models. Forward decay [8] supplies the idea behind the update rule.

2.2 TPNet: temporal walk matrices

Definition 2 (Temporal walk and score)**.** A k-step temporal walk from w_0 is $W = [(w_0, t_0), \ldots, (w_k, t_k)]$ with $t_0 > t_1 > \cdots > t_k$, where each (w_{i-1}, w_i) interacted at time t_i, in either stored direction. Its score at time $t \geq t_1$ is

$$s_t(W) = \prod_{i=1}^{k} e^{-\lambda(t-t_i)}, \qquad \lambda \geq 0. \tag{2}$$

The walk matrix $A^{(l)}(t) \in \mathbb{R}^{n \times n}$ has entries $A^{(l)}_{u,w}(t) = \sum_W s_t(W)$ over l-step walks from u to w using edges up to time t, and $A^{(0)} = I$.

A walk leaves its start over its newest edge and moves to strictly older edges. Two lemmas give the whole dynamics of A.

Lemma 1 (Aging). *With no event in $(t,t']$, $A^{(l)}(t') = e^{-\lambda l(t'-t)}A^{(l)}(t)$.*

Proof. The walk set is unchanged, and each of the l factors in (2) gains $e^{-\lambda(t'-t)}$. □

Lemma 2 (Event recursion). *Let edge (u,v,t) have a timestamp strictly above all earlier edges. Then for $l \geq 1$,*

$$A^{(l)}_{u,\cdot}(t^+) = A^{(l)}_{u,\cdot}(t) + A^{(l-1)}_{v,\cdot}(t), \qquad A^{(l)}_{v,\cdot}(t^+) = A^{(l)}_{v,\cdot}(t) + A^{(l-1)}_{u,\cdot}(t), \tag{3}$$

and all other rows are unchanged.

Proof. Times strictly decrease along a walk, so a walk using the new edge uses it exactly once and as its first hop. Hence new l-step walks from u are that edge followed by an $(l-1)$-step walk from v over strictly older edges, and likewise from v. The first hop has factor $e^{-\lambda(t-t)} = 1$, and the remaining factors equal the score of the shorter walk at t. No other start node gains a walk. □

2.3 TPNet: the sketch

Let $P \in \mathbb{R}^{n\times d_R}$ have independent $\mathcal{N}(0, 1/d_R)$ entries. TPNet stores $H^{(0)} = P$ and $H^{(l)} \in \mathbb{R}^{n\times d_R}$, initially zero for $l \geq 1$, and on (u,v,t) applies, for $l = k,\ldots,1$,

$$H^{(l)}_u \leftarrow H^{(l)}_u + e^{\lambda t}H^{(l-1)}_v, \qquad H^{(l)}_v \leftarrow H^{(l)}_v + e^{\lambda t}H^{(l-1)}_u. \tag{4}$$

The descending order of l makes each layer read the previous value of the layer below. The stored form carries a positive exponent although every walk weight decays. The next result resolves this.

Theorem 3 (Sketch identity). *Define $\widetilde{H}^{(l)}(t) := e^{-\lambda lt}H^{(l)}(t)$. If all timestamps differ, then for all l and t,*

$$\widetilde{H}^{(l)}(t) = A^{(l)}(t)\,P. \tag{5}$$

Proof. Induct over events. At the start both sides are zero for $l \geq 1$, and $\widetilde{H}^{(0)} = P = A^{(0)}P$ always. Between events $H^{(l)}$ is constant, so $\widetilde{H}^{(l)}(t') = e^{-\lambda l(t'-t)}\widetilde{H}^{(l)}(t)$, which matches Lemma 1. At event (u,v,t), multiply (4) by $e^{-\lambda lt}$:

$$\widetilde{H}^{(l)}_u(t^+) = \widetilde{H}^{(l)}_u(t) + e^{-\lambda lt}e^{\lambda t}H^{(l-1)}_v(t) = \widetilde{H}^{(l)}_u(t) + e^{-\lambda(l-1)t}H^{(l-1)}_v(t) = \widetilde{H}^{(l)}_u(t) + \widetilde{H}^{(l-1)}_v(t).$$

Using (5) before the event, this equals $\big(A^{(l)}_{u,\cdot}(t) + A^{(l-1)}_{v,\cdot}(t)\big)P$, which is the row in (3) times P. □

So old edges are not shrunk when stored. New edges are inflated relative to them, and the readout divides by the current inflation. For $l = 1$, $e^{-\lambda T}H^{(1)}_u(T) = \sum_i e^{-\lambda(T-t_i)}P_{v_i}$ over u's contacts. This is the forward-decay construction of Cormode et al. [8].

The positive form cannot run on our data. UN Vote spans 71 yearly steps, so $\lambda t \approx 2.2\times 10^4$ at $\lambda = 10^{-5}$, while a double overflows at e^{709}. The released code therefore stores $\widetilde{H}$ and ages it, multiplying layer l by $e^{-\lambda l\Delta}$ for a time step Δ. Section 4.7 deals with the cost.

Batches. The code applies edges in batches. For batch B with largest time T, it ages to T and then adds $e^{-\lambda(T-s)}\widetilde{H}^{(l-1)}$ for each edge at time s, using values from before the batch.

Proposition 4 (Batch semantics). *After a sequence of batches, $\widetilde{H}^{(l)} = \bar{A}^{(l)}P$, where $\bar{A}^{(l)}$ counts exactly the walks that use at most one edge from each batch, with strictly decreasing times between batches. Edges of equal time in one batch never chain, as the definition requires.*

Proof. Induct over batches. Each added message reads layer $l-1$ as it stood before the batch, so a new walk has one edge from this batch as first hop followed by a walk over earlier batches. Its first factor at T is $e^{-\lambda(T-s)}$, and the aged lower layer gives the remaining factors at T. □

When one yearly step forms one batch the walk matrix is exact. When a step is split across batches, edges of the same year can chain, which is wrong. When a 200-edge batch holds distinct times, walks inside the batch are lost at depth $l \geq 2$, a small approximation that TPNet shares.

For nodes u, v the model forms inner products among $\{r_u^{(l)}, r_v^{(l)}\}_{l=0}^{k}$, with $r_u^{(l)} = \widetilde{H}_u^{(l)}$, giving a pair feature of dimension $D = (2k+2)^2$. Each inner product is clamped at zero and passed through $\log(1+x)$ and an MLP. It joins neighbour and time information in an MLP-Mixer encoder and a link predictor [1].

3 Problems with existing work

Each family has a limit. Node memory is compact but does not by itself describe how two nodes relate. Walk sampling captures pair relations at the price of sampling for every query. Sequence models reach only limited structural depth. Supra-graph encodings need memory close to the full multi-layer graph, which limits scale.

TPNet avoids these limits, in time that does not depend on graph size in theory, but it leaves four problems open.

1. **One memory length.** A single rate λ gives every pair the same exponential memory. A real graph need not have an exponential memory, and short and long ranges share one scale.
2. **A rate tied to the time unit.** On yearly data with timestamps in seconds, $\lambda = 10^{-5}$ gives e^{-315} per year. What that memory actually does has not been examined.
3. **Cost in practice.** The analysis promises a per-edge cost independent of n, but the released code ages every row before each batch, for numerical safety.
4. **Same-time information.** Fixed-size batches split a yearly timestamp, so same-year edges reach the sketch before prediction.

The benchmarks have problems of their own. Random negatives are often true edges, 95.7% of them on UN Vote, and historical negatives give some positives away. Section 7 analyses both.

4 Our methods

4.1 Method 1: MuST-Walk

Choose rates $\lambda_1, \ldots, \lambda_J$ and maintain layers $H_j^{(l)}$ for each, with one shared $H_j^{(0)} = P$. Since (4) runs unchanged per scale, Theorem 3 holds per scale, so the extension adds no approximation beyond the projection. Tuned rates $\lambda^\star$ come from the TPNet configuration: 10^{-5} on Can. Parl. and UN Vote, 10^{-6} on US Legis. and Enron, 10^{-7} on UN Trade and UCI. The ladder is $\{10\lambda^\star, \lambda^\star, \lambda^\star/10, 0\}$ on UCI and Enron, and $\{10^{-5}, 10^{-7}, 2.2\times10^{-8}, 0\}$ on yearly data, with 10^{-6} added on US Legis. so that its tuned rate is present.

4.2 What several rates can express

Consider a depth-one feature that mixes scales with weights α_j. For a pair with contacts at ages δ_i,

$$\sum_j \alpha_j A_{\lambda_j}^{(1)}[u,v] = \sum_i \kappa(\delta_i), \qquad \kappa(\delta) = \sum_j \alpha_j e^{-\lambda_j \delta}. \tag{6}$$

A single scale fixes κ to one exponential. Several scales give a kernel in the span of exponentials, which is what a learned fusion can reach.

Proposition 5 (Expressivity on a grid)**.** *On a grid of step Δ let $x_j = e^{-\lambda_j \Delta}$. If the x_j are distinct, then for any target values on J consecutive ages there is a unique α with κ equal to them. If two x_j are numerically equal, those scales add nothing.*

Proof. On ages $a\Delta$, $a = 0, \ldots, J-1$, the system has matrix $W_{aj} = x_j^a$, a Vandermonde matrix with determinant $\prod_{i<j}(x_j - x_i)$. This is nonzero exactly when the x_j are distinct. □

Corollary 6 (The usable window of a rate)**.** *On data with smallest step Δ and span T, a rate with $\lambda T \ll 1$ behaves like $\lambda = 0$, since $1 - \lambda\delta \le e^{-\lambda\delta} \le 1$ for all ages $\delta \le T$. A rate with $\lambda\Delta \gg 1$ gives $x \approx 0$. Only rates with $1/T \lesssim \lambda \lesssim 1/\Delta$ are distinct.*

On yearly data the MuST ladder gives $x \approx \{0,\ 0.043,\ 0.5,\ 1\}$, four usable scales. The uniform ladder $\{10^{-4}, 10^{-5}, 10^{-6}, 10^{-7}\}$ gives $\{0,\ 0,\ 2\times10^{-14},\ 0.043\}$, about two. Half-lives $\ln 2/\lambda$ are 19.3 hours at 10^{-5}, 8 days at 10^{-6}, 80 days at 10^{-7} and one year at 2.2×10^{-8}.

4.3 Exact noise of the sketch

The pair features of Section 2.3 are sketched inner products, so we need their exact error.

Proposition 7 (Exact noise of a sketched inner product)**.** *For fixed $a, b, c, e \in \mathbb{R}^n$,*

$$\mathbb{E}\langle aP, bP\rangle = \langle a, b\rangle, \tag{7}$$

$$\mathrm{Var}\langle aP, bP\rangle = \tfrac{1}{d_R}\big(\|a\|^2\|b\|^2 + \langle a, b\rangle^2\big), \tag{8}$$

$$\mathrm{Cov}\big(\langle aP, bP\rangle, \langle cP, eP\rangle\big) = \tfrac{1}{d_R}\big(\langle a, c\rangle\langle b, e\rangle + \langle a, e\rangle\langle b, c\rangle\big). \tag{9}$$

Proof. Columns of P are independent. In one column, $X_a = \sum_i a_i P_{ic}$ and its analogues are jointly Gaussian, zero-mean, with $\mathrm{Cov}(X_a, X_b) = \langle a, b\rangle/d_R$. The inner product is a sum over d_R columns of $X_a X_b$, which gives (7). By Isserlis' theorem [16], $\mathrm{Cov}(X_a X_b, X_c X_e) = \mathrm{Cov}(X_a, X_c)\,\mathrm{Cov}(X_b, X_e) + \mathrm{Cov}(X_a, X_e)\,\mathrm{Cov}(X_b, X_c)$. Summing over columns gives (9), and $c = a, e = b$ gives (8). □

Corollary 8 (Resolution of one contact)**.** *Let $a = A^{(1)}_{u,:}$. The feature $\langle r^{(1)}_u, P_v\rangle$ estimates a_v with variance $(\|a\|^2 + a_v^2)/d_R$. If u has m contacts of equal weight, the signal-to-noise ratio for one of them is*

$$\rho = \sqrt{\frac{d_R}{m+1}}. \tag{10}$$

Proof. Take $b = e_v$ in (8). With unit weights $a_v = 1$ and $\|a\|^2 = m$, so $\rho = a_v/\sqrt{\mathrm{Var}} = \sqrt{d_R/(m+1)}$. □

The code sets $d_R = \min(\lfloor \ln 2E \rfloor \cdot 10, n)$, between 110 and 140 on our datasets. Corollary 8 therefore ties the sketch to graph structure: a node with about d_R contacts resolves one contact at about one standard deviation. Dense graphs sit there, as Section 5.1 shows.

4.4 Why one shared projection

Sharing P saves $J-1$ copies of $n\,d_R$ numbers, and it makes comparison across scales meaningful.

Proposition 9 (Inner products across scales)**.** *Let $r^{(l)}_u(\lambda) = A^{(l)}_\lambda[u,:]\,P$. With one shared P,*

$$\mathbb{E}\big\langle r^{(l)}_u(\lambda_a),\, r^{(m)}_v(\lambda_b)\big\rangle = \sum_w A^{(l)}_{\lambda_a}[u,w]\,A^{(m)}_{\lambda_b}[v,w], \tag{11}$$

a sum over shared intermediate nodes weighting u's walks at rate λ_a and v's at λ_b. With independent P_a, P_b the expectation is 0.

Proof. The first claim is (7). For independent zero-mean projections $\mathbb{E}[P_a P_b^\top] = 0$. □

Sharing has a price, which (9) states exactly: noise across scales is correlated, and nearly equal scales produce nearly equal features and nearly equal noise, consistent with Proposition 5.

We can now state the guarantee across scales.

Theorem 10 (Projection guarantee for J scales)**.** *Let $\varepsilon \in (0, \frac{1}{2}]$ and $N = J(k+1)n$. If $d_R \ge 24\,\varepsilon^{-2}\log\big(4^{1/3}N\big)$, then at any fixed time, with probability at least $1 - 1/N$, every pair x, y among the N rows of all layers and scales satisfies $|\langle xP, yP\rangle - \langle x, y\rangle| \le \varepsilon\|x\|\,\|y\|$.*

Proof. The rows depend only on the stream, not on P. For a unit vector z, $d_R\|zP\|^2 \sim \chi^2_{d_R}$, and the Chernoff bound gives $\Pr\big(|\|zP\|^2-1|>\varepsilon\big) \le 2\exp\big(-\frac{d_R}{2}(\frac{\varepsilon^2}{2}-\frac{\varepsilon^3}{3})\big) \le 2e^{-d_R\varepsilon^2/6}$, using $\varepsilon \le \frac{1}{2}$. For unit x, y, polarisation gives $\langle x,y\rangle = \frac{1}{4}(\|x+y\|^2-\|x-y\|^2)$, and if both norms are preserved up to factor $1\pm\varepsilon$ the error is at most $\frac{\varepsilon}{4}(\|x+y\|^2+\|x-y\|^2)=\varepsilon$. A union bound over at most N^2 pairs and two norms per pair gives failure probability at most $4N^2e^{-d_R\varepsilon^2/6}$. The assumption says $d_R \ge 8\varepsilon^{-2}\log(4N^3)$, so this is at most $4N^2(4N^3)^{-4/3} \le 1/N$. □

This is a Johnson–Lindenstrauss argument [17], and TPNet states the single-scale version [1]. Moving from TPNet's $N=(k+1)n$ to $J(k+1)n$ adds only $\log J$. The cap $d_R \le n$ and the width in use matter more: for $\varepsilon=\frac{1}{2}$ the theorem asks for $d_R \approx 810$ to 1,040 on our datasets, against 110 to 140 in the code. At the width in use the bound corresponds to ε between 1.21 and 1.53, so the guarantee is vacuous and Proposition 7 is the operative result.

4.5 Fusion

Let $f_j \in \mathbb{R}^D$ be the per-scale pair features and f_b the feature at the tuned rate. Concatenation, $f=[f_1,\dots,f_J] \in \mathbb{R}^{JD}$, is our main configuration. It contains the tuned-rate feature, so in principle it loses nothing. Gated residual fusion is

$$f = f_b + \beta \sum_{j=1}^{J} \alpha_j f_j, \qquad \alpha = \mathrm{softmax}\big(g_\phi([f_1,\dots,f_J])\big), \quad \beta_0 = 0, \tag{12}$$

so training starts exactly at TPNet, and average fusion fixes $\alpha_j = 1/J$.

4.6 Method 2: MuST-HN

MuST-HN is MuST-Walk with historical-negative training. We replace about half of the fake training pairs with pairs seen strictly before the batch and not active in its interval, and we select the best epoch on historical validation. Historical negatives come from [3]. Training on them is not a new idea; see ENS [11], CurNM [12] and ATNSF [13].

4.7 Efficiency: the lazy update

Equation (4) touches $2k$ vectors per edge, so $O(kd_R)$ per edge. Ageing must touch every row, because every row ages. For b edges per batch,

$$T_{\text{eager}} = O\big(Jkd_R(n+b)\big) \text{ per batch} = O\big(Jkd_R(1+n/b)\big) \text{ per edge}, \tag{13}$$

which grows with n.

Store $\hat{H}^{(l)}$ and a scalar c_l with $c_0=1$, keeping the invariant $\widetilde{H}^{(l)} = c_l\hat{H}^{(l)}$.

Proposition 11 (Lazy update)**.** *Aging by Δ with $c_l \leftarrow c_l e^{-\lambda l\Delta}$, and an event of weight w with*

$$\hat{H}^{(l)}_u \leftarrow \hat{H}^{(l)}_u + w\,\frac{c_{l-1}}{c_l}\,\hat{H}^{(l-1)}_v, \tag{14}$$

preserve the invariant and reproduce the decay form exactly in real arithmetic. If c_l falls below $\theta = 10^{-12}$ the layer is written out and c_l reset to 1. With $\bar{\Delta}$ the mean time gap per batch, the amortised cost is

$$T_{lazy} = O(Jkd_R) + O\Big(Jk^2\,\frac{nd_R}{b}\,\lambda\bar{\Delta}\Big) \text{ per edge}. \tag{15}$$

Proof. Aging: c_l takes the factor of Lemma 1 and $\hat{H}$ is unchanged. Event: multiply (14) by c_l to get $\widetilde{H}^{(l)}_u \leftarrow \widetilde{H}^{(l)}_u + w\widetilde{H}^{(l-1)}_v$. Write-out: $\hat{H}^{(l)} \leftarrow c_l\hat{H}^{(l)}$ and $c_l \leftarrow 1$ leave the product fixed. Layer l loses a factor $\lambda l\bar{\Delta}$ in log scale per batch, so it is written every $\ln(1/\theta)/(\lambda l\bar{\Delta})$ batches at cost $O(nd_R)$. Dividing by b edges per batch and summing over $l \le k$ and J scales gives the second term, since $\sum_{l\le k} l = O(k^2)$. □

Without renormalisation $c_{l-1}/c_l = e^{+\lambda(t-t_0)}$, so the positive exponent of (4) returns relative to a moving origin t_0. Only the second term in (15) grows with n, and it is small when $\lambda\bar{\Delta} \ll 1$. Values match the eager form to about 10^{-7} in our tests.

Operation	Cost	Note
Update, eager	$O(Jk\,d_R(1+n/b))$ per edge	released code path
Update, lazy	$O(Jk\,d_R)$ per edge, plus renormalisation	ours
Strict buffer	$O(1)$ amortised per edge	each edge moved once
Pair feature	$O(J(2k+2)^2 d_R)$ per query	$(2k+2)^2$ inner products per scale
Sketch state	$(1+Jk)\,n\,d_R$ numbers	P stored once
Rebuild from scratch	$O(Jk\,d_R\,E)$ for E edges	what streaming avoids

Table 1: Time and space. k walk depth, J scales, n nodes, d_R sketch width, b batch size.

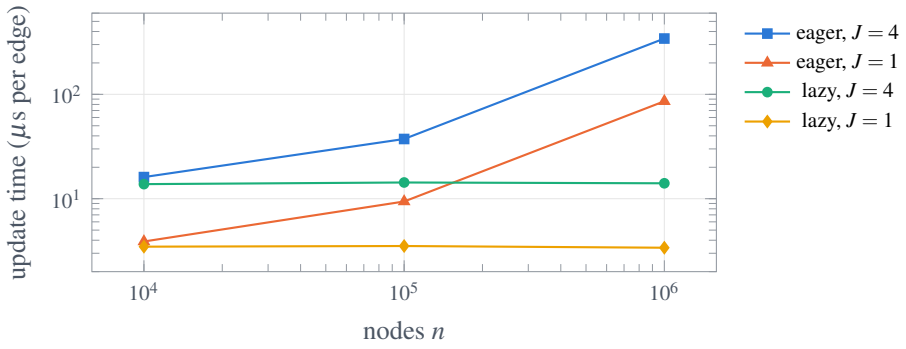


Figure 1: Update cost on a Tesla T4, batch size 200. Gaps of about 100 s per batch mean no renormalisation occurs in the benchmark's 220 batches, so the curves show the first term of (15) only.

Measured efficiency. Eager cost grows with n and lazy cost stays flat, as (13) and (15) predict. Lazy is 1.2, 2.6 and 24.4 times faster at 10^4, 10^5 and 10^6 nodes for $J = 4$. Our DyGLib datasets have at most 1,899 nodes and the largest TGB dataset has 352,637, so the 24.4 figure describes a size we benchmarked and did not train on. At 10^6 nodes, lazy $J = 4$ against $J = 1$ costs 4.1 times the update time, 4.0 times the pair-feature time and 3.25 times the state. The state formula is exact: at $k = 3$, $d_R = 160$, $n = 10^6$ it predicts 2,441.4 and 7,934.6 MiB, and we measured 2,441.5 and 7,935.1 MiB. Streaming one batch against a full rebuild at $n = 10^5$ with 10^6 past edges takes 2.47 ms against 11.93 s for $J = 4$ on an RTX 6000 Ada, a factor of about 4,800.

Minutes per run, mean of 3 seeds	US Legis.	Can. Parl.	UN Trade	UCI	Enron
TPNet	14	23	170	17	30
MuST-Walk	39	40	415	46	60
MuST-HN	26	27	219	40	60

Table 2: Whole-run time on Tesla T4s, including evaluation, from run logs. Early stopping changes epoch counts, so these are indicative. UN Vote is left out because its runs used mixed hardware.

End to end, MuST-Walk took 1.7 to 2.8 times as long per run as TPNet. On tgbl-review, final evaluation took about 2.75 hours for TPNet and 5.0 for MuST-Walk on a T4.

4.8 Correctness: the strict buffer

The released code updates the sketch after each 200-edge batch. One yearly timestamp holds thousands of edges across many batches, so when batch 2 of year y is predicted the sketch already contains batch 1 of that year. On the four yearly datasets 96–99.9% of test batches are affected, and these edges carry 85–99% of the

direct-connection weight. Zhang et al. [9] reported this first, and Lampert et al. [10] describe the general batching problem. We confirmed it independently. Our fix holds incoming edges in a buffer, and before predicting a batch with earliest time $t_{\min}$ it applies exactly the waiting edges older than $t_{\min}$.

Proposition 12 (Strict invariant)**.** *With the buffer, at any query time t the sketch contains an edge at time s only if $s < t$, and it contains every observed edge with $s < t_{\min}$. All waiting edges of one past step are applied together, so by Proposition 4 such a step never chains with itself. With no tied timestamps the buffer leaves the sketch equal to the original.*

Proof. Edges enter only when $s < t_{\min} \le t$, and the rule applies all such edges. Edges of a past step are all older than the first batch lying wholly in a later step, so they enter in one application. Each edge moves once, hence $O(1)$ amortised. With distinct times, the edges of the previous batch are all older than $t_{\min}$, so the state at each prediction matches the original. □

TPNet, historical AP, three seeds	US Legis.	Can. Parl.	UN Trade	UN Vote
Original code, 30 epochs	91.11	83.43	71.98	74.94
Fixed code, 100 epochs	66.25	81.37	61.00	57.18

Table 3: The baseline before and after correction. Code and training length both changed, so this gap is not the leak alone.

One pilot isolates the leak. On US Legis., seed 0, at 100 epochs in both arms, removing it takes TPNet from 93.41 to 67.19 and the ladder from 93.79 to 73.81. So published scores on these datasets partly reflect same-year edges.

4.9 Correctness: what the sketch reads at query time

Equation (2) scores a walk at the query time. The code does not: it ages the sketch only when applying edges, and a read applies no decay.

Proposition 13 (Read weight)**.** *Let $\tau \le t$ be the time of the last applied edge. The model reads $A^{(l)}(\tau)P$, not $A^{(l)}(t)P$, so a direct edge at time s has read weight $e^{-\lambda(\tau - s)}$. With the strict buffer on a grid of step Δ, $\tau = t - \Delta$ outside batches that straddle two steps, so an edge $a \ge 1$ steps old has read weight*

$$w(a) = e^{-\lambda(a-1)\Delta}. \tag{16}$$

Proof. Aging happens only inside the update, which sets the clock to the largest applied time, and a read returns stored layers. By Proposition 12 the clock stands at $t - \Delta$ during step t. Substitute $s = t - a\Delta$. □

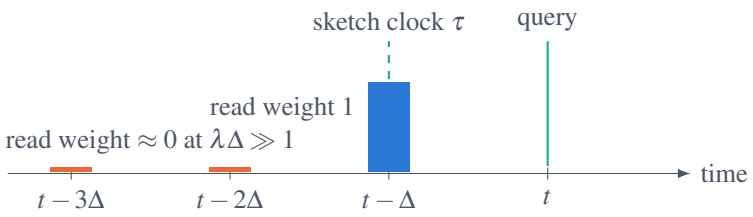


Figure 2: Yearly data under the strict buffer. The clock stops at the last applied step, so the previous step keeps weight 1 and decay starts from the step before it.

Dataset	$\lambda^\star$	$e^{-\lambda^\star \Delta}$	$w(1)$	$w(2)$	AUC
US Legis.	10^{-6}	2×10^{-14}	1	2×10^{-14}	0.772
Can. Parl.	10^{-5}	10^{-137}	1	0 in float32	0.966
UN Trade	10^{-7}	0.043	1	0.043	0.745
UN Vote	10^{-5}	10^{-137}	1	0 in float32	0.610

Table 4: Yearly data, $\Delta = 86400 \times 365$ s. Read weights measured on the real eager and lazy modules. AUC: on real data, at the first test step and the tuned rate, $\langle r_u^{(1)}, r_v^{(0)} \rangle$ separating 3,000 pairs active one step ago from 3,000 seen only earlier. An erased step would give a feature near 0.

So on yearly data the tuned rate does not erase the previous step. It keeps that step at full weight and drops older ones on US Legis., Can. Parl. and UN Vote. On UCI and Enron the clock lags by under a batch and the effect is negligible. All our results use the code as it ran, so (16) is the memory these models had.

5 Experimental setup

5.1 Data and graph structure

We use six DyGLib datasets [4], opened pair by pair, and four TGB datasets [2] evaluated without pair-level inspection of tgbl-wiki and tgbl-review. Four DyGLib graphs are yearly snapshots of institutions and two are continuous message streams.

Dataset	Nodes	Edges	Steps	Density	Snapshot	Degree	Gini	Transitivity	Edges/pair	d_R
US Legis.	225	60,396	12	0.661	0.200	148.2	0.15	0.81	3.6	110
Can. Parl.	734	74,478	14	0.170	0.019	123.9	0.45	0.44	1.6	110
UN Trade	255	507,497	32	0.639	0.307	161.5	0.22	0.82	24.5	130
UN Vote	201	1,035,742	72	0.981	0.552	196.2	0.01	0.99	52.5	140
UCI	1,899	59,835	58,911	0.008	0.0004	14.6	0.65	0.06	4.3	110
Enron	184	125,235	22,632	0.132	0.005	22.8	0.35	0.37	56.5	120

Table 5: Static structure. Density: share of node pairs that ever interact. Snapshot: mean share active in one step, or one week on UCI and Enron. Degree: mean distinct partners. Gini: inequality of degree.

Dataset	Persistence	Node stay	New-node edges	Never seen	Free positives	Contacts	Separation ρ
US Legis.	0.786	0.886	22.7%	23.1%	34.3%	99.0	1.06
Can. Parl.	0.297	0.920	1.8%	45.4%	51.3%	10.0	2.29
UN Trade	0.884	0.986	0.0%	1.0%	2.0%	90.4	0.92
UN Vote	0.991	0.996	0.0%	0.0%	0.3%	171.5	0.81
UCI	0.209	0.493	22.9%	35.1%	42.1%	3.7	2.34
Enron	0.468	0.809	1.5%	9.5%	11.2%	2.1	2.83

Table 6: Temporal structure, on test edges where it applies. Persistence: chance that a pair active in one period is active in the next (step, or week on UCI and Enron). Node stay: chance a node stays active into the next period (step, or 30 days). Never seen: positives with no contact in either direction before their batch, which is what the sketch and the test can know; counted edge by edge, the UCI figure would be lower because pairs repeat inside a batch. Free positives: never seen in stored direction (Proposition 15). Contacts: median effective contacts $\|a\|_1^2/\|a\|_2^2$ at the tuned rate. Separation: median $a_v\sqrt{d_R}/\|a\|$ for test positives.

Table 6 confirms Corollary 8. On the dense graphs effective contacts match or exceed d_R and one contact sits near one standard deviation of noise; on the sparse graphs it sits at 2.3 to 2.8. Common-neighbour features saturate the same way. On UN Vote and UN Trade every test pair, real or fake, has a common neighbour, while the median count on UN Vote is 197 of 200 for both classes.

The memory of each graph. Figure 3 shows the empirical memory kernel: the chance that a pair is active given the age of its last contact. This is the curve a decay kernel tries to match. Three shapes appear. US Legis. has a cliff: 0.77 at one step, 0.015 at two, and zero from three on, consistent with cohorts of legislators changing. UN Trade and UN Vote have long tails, since the same countries stay related for decades. Can. Parl. is low and flat after one step. On the streams, return probability falls smoothly from under a day to months, and much faster on UCI than on Enron. UCI pairs return after a median 20 minutes, with a 90th percentile of 2.5 days, against 22.5 hours and 21 days on Enron.

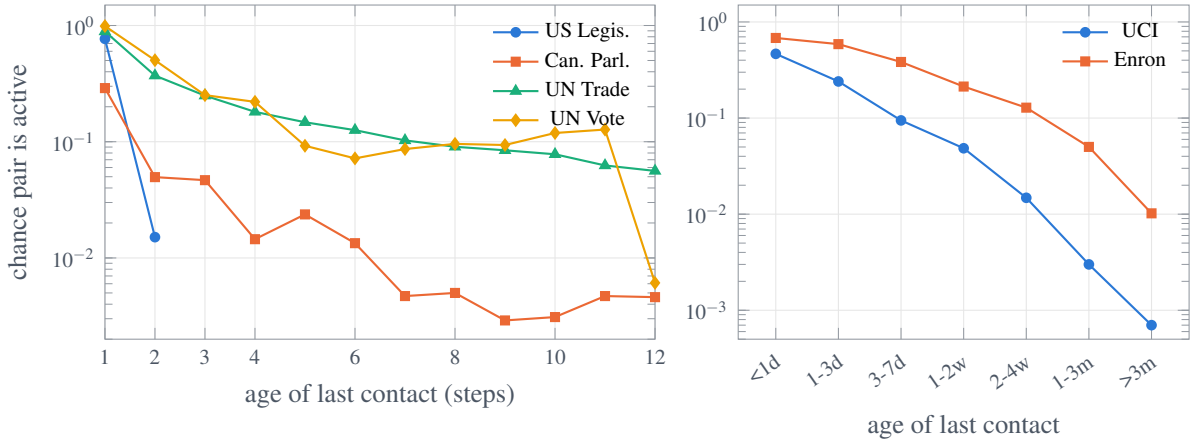


Figure 3: Empirical memory kernel, log scale. Left: chance of being active at a step, from 30% of the way through the data. Right: chance of contact within the next week, weekly anchors. US Legis. stops at age 2 because the chance is exactly 0 from age 3.

Raw edge features are unscaled, with a maximum of 52,504,548 on UN Trade. 18.0–31.8% of pre-validation edges touch held-out nodes and never enter the sketch during training, on all six datasets.

5.2 Protocol

Training. Strict time and lazy updates, 100 epochs, patience 20, seeds 0–2, chronological 70/15/15 split. Test types are random, historical (primary) and inductive.

Model selection. We keep the epoch with the best validation score and test it once; the test metrics printed each epoch were ignored. TPNet and MuST-Walk select on random validation, and MuST-HN on historical validation.

TGB. MRR with the official evaluator on tgbl-wiki (999 negatives), tgbl-review (100), tgbl-uci and tgbl-enron; 30 epochs, patience 5; per-epoch validation on the first 20% of validation edges in time order, then full validation and test. This is not the authors' protocol, so these are not leaderboard entries. As a sanity check, TPNet on tgbl-wiki gave 0.8239, within the pre-registered 0.02 of its published 0.827.

Decision rule. Rules were written before results. Per dataset, a comparison passes if the mean gain is at least $+1.0$ AP and at least 2 of 3 seeds win (TGB: $+0.005$ MRR). Across datasets we planned an exact one-sided Wilcoxon test [14] with Holm correction [15] over the four comparisons of the wider experiment.

A flaw in that test. With n paired datasets the smallest one-sided p is 2^{-n}, so with six datasets and four tests the smallest adjusted p is $4 \cdot 2^{-6} = 0.0625 > 0.05$. No comparison could pass. We noticed this after the runs and did not change the rule.

Feature-level analysis. To test mechanisms without a neural model, we stream exact walk matrices with the strict read, form five walk features per rate (direct contact, two-hop walks, common neighbours and each node's activity), and fit logistic regression on the validation period to predict the test period, on five negative samples. Separately we push the same features through a real projection of width d_R. This uses all edges before validation, so it is idealised, and it is linear. It is exploratory and was added after the main analysis.

6 Results

6.1 Main comparison

		MuST-Walk			MuST-HN		
Dataset	TPNet	AP	Gain	Seeds	AP	Gain	Seeds
US Legis.	66.25±10.86	75.90±4.28	**+9.64**	2/3	90.54±0.57	**+24.28**	3/3
Can. Parl.	81.37±4.39	86.11±3.87	**+4.74**	3/3	91.32±0.40	**+9.95**	3/3
UN Trade	61.00±1.20	60.21±0.97	−0.79	2/3	87.48±0.11	**+26.48**	3/3
UN Vote	57.18±4.22	59.89±0.91	**+2.72**	2/3	71.95±1.00	**+14.77**	3/3
UCI	86.57±0.61	85.55±0.18	−1.02	0/3	96.31±0.26	**+9.74**	3/3
Enron	81.51±1.26	79.16±0.78	−2.35	0/3	90.52±0.05	**+9.01**	3/3

Table 7: Historical test AP, mean ± std over three seeds, with gain over TPNet and seeds won against it. Bold marks a pass under the per-dataset rule. UN Vote MuST-Walk seeds 1, 2 and MuST-HN seeds 1, 2 hit time limits at 52 and 28 epochs and were tested at their best checkpoint.

MuST-Walk passes on 3 of 6 datasets and wins 9 of 18 seeds. Its pre-registered cross-dataset test gives adjusted $p = 0.4688$, so it is not supported. MuST-HN against TPNet was not a pre-registered comparison, so we report it as exploratory: it passes on 6 of 6 datasets and wins 18 of 18 seeds. Its gain comes from the training change, not from multi-scale: a single-rate model given the same training and selection scores about the same (−0.25 AP for MuST-HN, which wins 7 of 18 seeds). The gain over TPNet should therefore not be read as a gain from several decay rates.

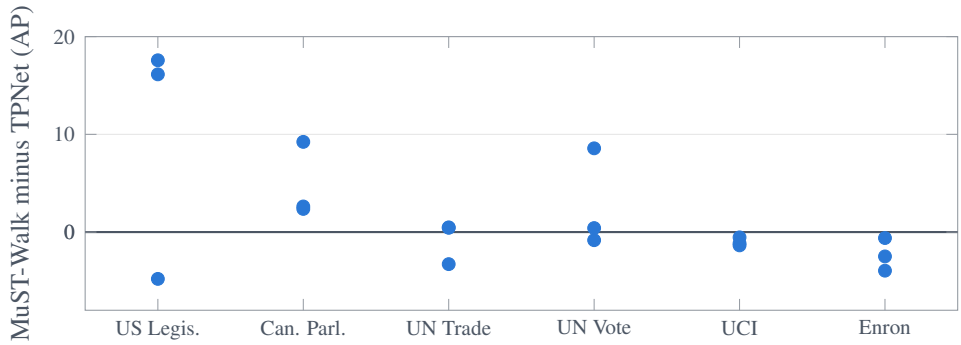


Figure 4: Each dot is one seed. The spread on US Legis. and UN Vote shows why three seeds cannot settle those means: one seed carries the UN Vote gain, and US Legis. runs from −4.79 to +17.58.

Figure 4 shows why the pass count needs care. Modelling each arm as normal with its observed seed spread and no true difference, the rule passes with probability 0.39, 0.35, 0.13, 0.32, 0.00 and 0.12 across the six datasets. Noise alone would pass about 1.3 datasets, and 3 or more about 11% of the time. This is rough, since three seeds estimate a spread poorly. Random validation adds to the spread: two UN Vote TPNet seeds had validation 56.57 and scored 60.25 and 52.37, and on Can. Parl. later selected epochs score worse (rank correlation −0.60 over 6 runs).

AP gain over TPNet	US Legis.	Can. Parl.	UN Trade	UN Vote	UCI	Enron
MuST-Walk, random	−0.63	−0.73	+0.41	+0.53	−0.46	+0.09
MuST-Walk, inductive	+3.99	+1.38	+1.87	+2.80	−1.32	+2.07
MuST-HN, random	+0.78	−3.54	−6.15	+0.39	−4.95	−1.02
MuST-HN, inductive	+10.72	+2.23	+26.62	+8.87	+15.27	+10.97

Table 8: Secondary metrics. MuST-Walk shows no gain on random tests (mean −0.13) and gains on 5 of 6 datasets on inductive tests (mean +1.80). MuST-HN costs random AP on 4 of 6 datasets and raises inductive AP on all six.

6.2 TGB datasets

Test MRR	TPNet, no strict	TPNet	MuST-Walk	MuST-HN
tgbl-wiki	0.8251	0.8239±0.0034	0.8260±0.0029	0.7673±0.0076
tgbl-review	0.4157	0.4033±0.0033	0.4025±0.0027	0.4939±0.0019
tgbl-enron	0.636	0.5909±0.0011	0.5837±0.0064	0.5854±0.0128
tgbl-uci	0.283	0.2620±0.0724	0.3868±0.0045	0.3192±0.0344

Table 9: Mean ± std over three seeds; no strict is seed 0 only. tgbl-review runs were cut by time limits: TPNet 6–9 epochs, MuST-Walk and MuST-HN 5–6. tgbl-wiki MuST-Walk seed 1 was rerun after a 2-epoch stop, by a rule fixed before the rerun: 0.8158, then 0.8228.

MuST-Walk passes against TPNet on tgbl-uci only (+0.1248, 3 of 3 seeds), where TPNet seed 0 is low at 0.1784; on seeds 1 and 2 the gain is +0.0871 and +0.0753. MuST-HN against TPNet is +0.091 on tgbl-review and +0.057 on tgbl-uci, both on all seeds, and −0.057 on tgbl-wiki and −0.005 on tgbl-enron. TGB negatives are fixed in advance and are not all historical, so Proposition 15 need not apply there, which fits the mixed result; we did not test this directly. With four datasets this is descriptive, as pre-registered.

6.3 Ablations

Scale count, decay set and fusion ran on US Legis. on corrected code, as pre-registered, so they do not cover other graphs. We show each against TPNet at 66.25.

Configuration	Historical AP	Gain over TPNet	Seeds won
$J = 5$, concatenation (MuST-Walk)	75.90±4.28	+9.64	2 of 3
$J = 3$	75.21±6.84	+8.96	2 of 3
Uniform ladder $\{10^{-4}, 10^{-5}, 10^{-6}, 10^{-7}\}$	69.40±2.51	+3.15	2 of 3
Log-spaced half-lives	70.72±1.83	+4.47	2 of 3
Average fusion	80.88±2.20	+14.63	3 of 3
Gate fusion	81.72±3.29	+15.47	3 of 3
Gate with per-scale LayerNorm	72.59±5.22	+6.34	2 of 3

Table 10: Fixed code, US Legis., three seeds. Descriptive.

The uniform ladder gains least, which fits Proposition 5: its scales collapse to about two values on yearly data and it has no slow scale, so it cannot break the tie of Proposition 16. Gate and average have the largest gains, possibly because they use D rather than JD inputs. The learned β stayed small (-0.04 to $+0.03$), so the gate is not a strong switch. On a task built to need two time scales, MuST-Walk beat the best single rate by $+2.69$ and $+2.21$ in two seeds, which shows the mechanism can work. Before the correction, a pre-registered confirmation of the four-rate ladder on new seeds 3–4 of four yearly datasets gave -0.57 AP, 4 of 8 wins, $p = 0.63$: not supported.

7 Results explained

7.1 Random negatives on near-complete graphs

Proposition 14 (Reference level for random AP)**.** *Let there be N positives and N negatives, of which a fraction f are true edges at that time. A scorer that ranks every true edge above every non-edge and cannot rank among true edges has* $\mathrm{AP} = 1/(1+f)$.

Proof. All $N(1+f)$ true edges share the top score, so in (1) the first step has recall 1 and precision $N/(N+fN)$. □

	US Legis.	Can. Parl.	UN Trade	UN Vote	UCI	Enron
Fake pairs that are real, f	44.0%	8.3%	59.9%	95.7%	0.1%	5.3%
$1/(1+f)$	69.4	92.3	62.5	51.1	99.9	95.0
TPNet, random AP	69.71	80.55	67.53	55.23	97.33	93.26

Table 11: This is a reference level, not a hard limit: a model can pass it by ranking test pairs above other true pairs, by up to about 5.5 points here. On US Legis., UN Trade and UN Vote every model sits this close, so random AP says little there, and so does validation on it.

7.2 Historical negatives give some positives away

A historical negative must have been seen in its stored direction. That simple fact has a large consequence.

Proposition 15 (Free positives)**.** *Let g be the share of test positives never seen in stored direction. Each such pair is certainly positive under the historical test. A scorer using only this fact has* $\mathrm{AP} = (1+g)/2$ *on a balanced test.*

Proof. No historical negative has this property, so these gN pairs score above everything, with precision 1 up to recall g. All other $(1-g)N$ positives tie with the N negatives, so the next step reaches recall 1 at precision $\frac{1}{2}$. Then (1) gives $g+(1-g)/2$. □

The floor is 67.1 on US Legis., 75.7 on Can. Parl., 71.0 on UCI and 55.6 on Enron, with no learning at all. This has a sign consequence. Under random negatives familiar pairs are likely real, so a model learns that history means positive. Under historical negatives every fake is familiar, so novelty means positive. All-time contact count at $\lambda = 0$ has AUC 0.35 on US Legis. and Can. Parl., below chance.

Proposition 16 (Fast decay ties novelty with absence)**.** *On a grid with $\lambda\Delta \gg 1$, by (16) a pair whose last contact is $a \geq 2$ steps old has direct read $A^{(1)}_{u,v} \approx 0$, the same as a never-seen pair. No function of that feature can separate them. At $\lambda = 0$ a seen pair has $A^{(1)}_{u,v} \geq 1$ and a never-seen pair has 0, so a slow scale separates them exactly.*

Proof. For the first claim, the contacts of the pair each contribute at most $e^{-\lambda(a-1)\Delta}$, summing to a value near 0. At $\lambda = 0$ each contact contributes 1. □

Proposition 16 is what several rates add. At the tuned rate, 77.7% of historical negatives and 24.7% of positives on US Legis. read as zero, and 88.3% and 53.3% on Can. Parl. On UCI no negative reads as zero, because its tuned half-life is 80 days. Novelty is already visible there.

7.3 Sketches cannot see direction

No historical negative is active in its stored direction, but many are active in reverse: 81.0% on UN Vote, 23.2% on UN Trade, 12.7% on US Legis. and under 2% elsewhere.

Proposition 17 (Direction blindness)**.** *The sketch state is invariant under reversing the stored direction of any edge.*

Proof. Equation (4) treats u and v symmetrically: the pair of updates is unchanged by swapping them. □

So the walk feature of a positive and of a same-time reversed negative contain the same pairs of rows, and only the neighbour encoder can use direction. On relations with no natural direction, such as co-voting, the stored order carries little meaning, yet the test depends on it.

Historical test AP	US Legis.	Can. Parl.	UN Trade	UN Vote	UCI	Enron
Recency rule, direction-blind	71.76	66.56	88.99	55.05	78.56	85.91
Recency rule, direction-aware	69.96	67.18	94.58	96.27	75.07	86.32
Novelty-aware rule	89.59	93.92	84.58	86.32	97.36	93.20
Floor $(1+g)/2$	67.1	75.7	51.0	50.1	71.0	55.6

Table 12: No-learning rules, exploratory. Recency rules use geometrically weighted recency on yearly data and last-seen time on UCI and Enron. The novelty-aware rule ranks pairs never seen in stored direction first, then by last contact in that direction, and is measured on our replication of the test, where the blind last-contact rule gives 70.78, 64.56, 73.52, 54.84, 78.48 and 85.53.

7.4 Which decay rates carry signal

Figure 5 takes the neural model away and asks how much historical-test signal each rate carries in walk features alone. The picture differs by graph, and it matches the memory kernels of Figure 3. On US Legis. and Can. Parl. the curve rises toward slow rates and the ladder sits well above any single rate. On UN Trade it is flat: any rate works and the ladder adds nothing. On UN Vote everything is near chance. On UCI and Enron it peaks at a fast rate and falls toward no decay, as bursty contact predicts.

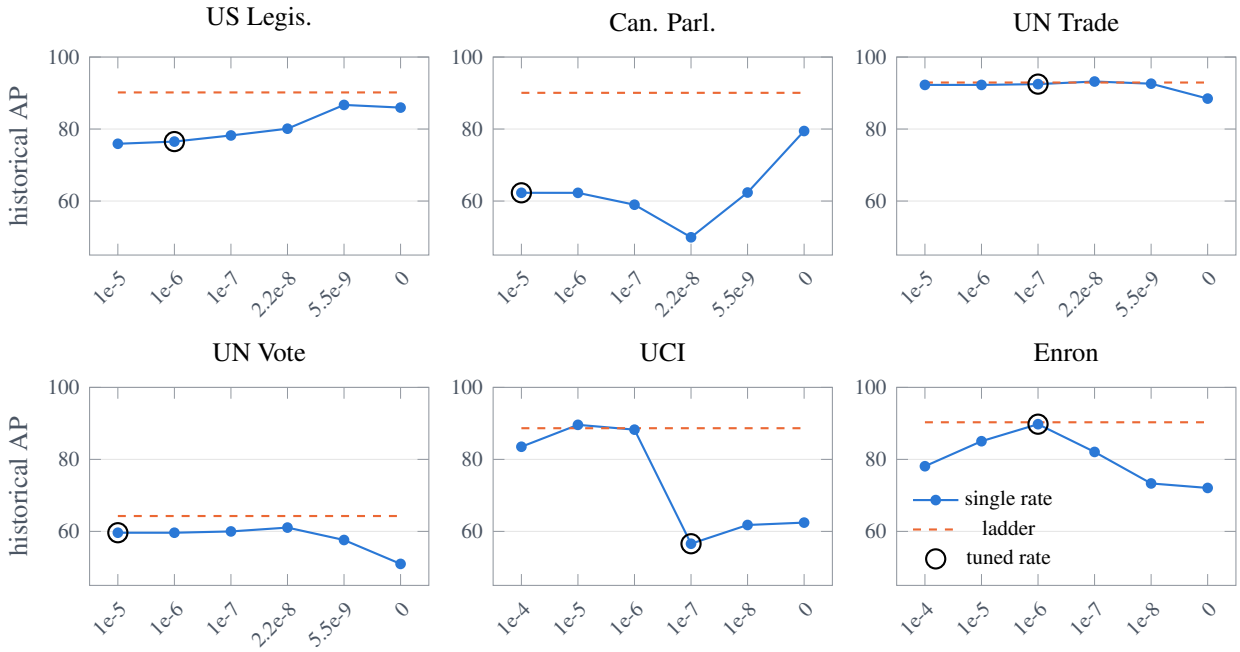


Figure 5: Feature-level historical AP by decay rate, faster on the left: logistic regression on exact walk features, trained on the validation period with historical negatives, mean of five negative samples. The ring marks the tuned rate, the dashed line the ladder.

	All test positives			Free positives removed		Ladder gain	
Dataset	Tuned	Best single	Ladder	Tuned	Ladder	All	Removed
US Legis.	76.52	86.70	90.17	82.13	83.45	+13.65	+1.32
Can. Parl.	62.29	79.45	90.06	78.83	84.01	+27.77	+5.18
UN Trade	92.48	93.20	92.92	93.06	93.83	+0.44	+0.77
UN Vote	59.63	61.06	64.27	61.54	65.16	+4.64	+3.62
UCI	56.57	89.60	88.66	84.56	77.97	+32.09	−6.59
Enron	89.78	89.78	90.31	92.19	92.27	+0.53	+0.08

Table 13: Feature-level test of the mechanism. Best single: rate chosen by cross-validation on the validation period. Free positives removed: positives limited to pairs seen in stored direction, so each could also have been drawn as a negative, with negatives rebalanced. Exploratory.

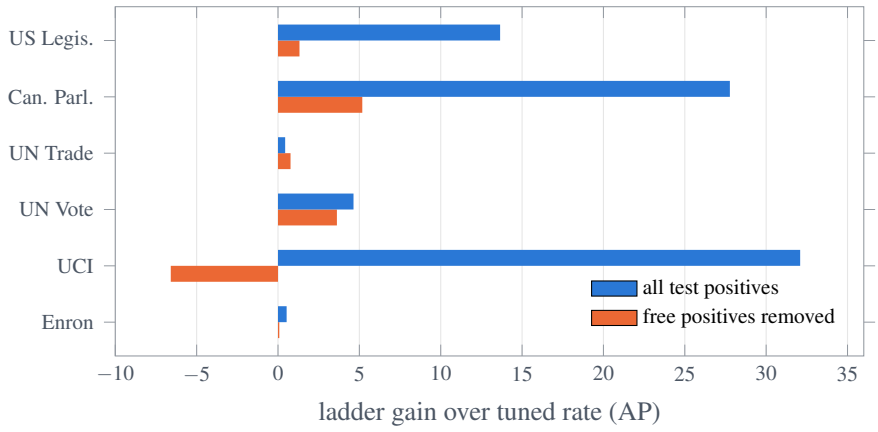


Figure 6: Remove the free positives and almost all of the large ladder gains disappear, so what several rates add on these graphs is mostly the ability to spot pairs that cannot be negatives.

Table 13 and Figure 6 are the central result. With free positives removed, the ladder's gain falls from +13.65 to +1.32 on US Legis. and from +27.77 to +5.18 on Can. Parl. On UCI it turns negative, because this ladder omits the 10^{-5} rate the graph needs. What remains is modest real signal, largest on Can. Parl. and UN Vote. The novelty-aware rule of Table 12 confirms it from the other side: with no learning it beats both proposed methods on Can. Parl., UN Vote, UCI and Enron.

Historical AP	US Legis.	Can. Parl.	UN Trade	UN Vote	UCI	Enron
Walk features, tuned, random training	73.79	72.79	72.03	49.31	65.91	85.43
TPNet	66.25	81.37	61.00	57.18	86.57	81.51
Walk features, ladder, random training	79.20	78.83	70.81	60.00	80.03	78.23
MuST-Walk	75.90	86.11	60.21	59.89	85.55	79.16
Walk features, ladder, historical training	90.17	90.06	92.92	64.27	88.66	90.31
MuST-HN	90.54	91.32	87.48	71.95	96.31	90.52

Table 14: Linear walk features against the neural models. The ladder minus tuned gain has the same sign as MuST-Walk minus TPNet on 5 of 6 datasets, UCI being the exception, so the neural gains are not pure seed noise. Historical training lifts both.

Table 14 ties this to the neural results. A linear model on walk features tracks the neural models, and on US Legis., Can. Parl. and Enron the historically trained ladder comes within 1.3 points of MuST-HN with no neural network. It also tempers the noise argument of Section 6: the direction of multi-scale gains is largely real at the feature level, although three seeds cannot size it. Whether the neural MuST-Walk uses novelty in exactly this way is likely, not proven, since we did not inspect its decisions.

7.5 Dataset by dataset

The numbers below are measured. Readings of what the neural models do are likely explanations, since we tested them at feature level and not inside the trained models.

US Legis.: turnover creates free positives. Cohorts change each Congress, so 22.7% of test edges involve a new node and 34.3% of positives are free. The memory kernel has a cliff, so a pair idle for two steps is effectively gone. At the tuned rate such a pair reads zero, tying with new pairs (Proposition 16). A slow scale breaks the tie, which fits the +9.64. But slow memory has no forecasting value here, since the chance of return is 0 from age 3. Without free positives the gain is +1.32. The model is exploiting the test, not long memory.

Can. Parl.: new pairs among known nodes. Only 1.8% of test edges have a new node, yet 45.4% of positives are new pairs. The graph is sparse in each step, with snapshot density 0.019 and persistence 0.297, so members keep forming new relations. Two-hop structure matters here more than anywhere else: at the tuned rate common neighbours reach AUC 0.87 against 0.65 for direct contact, as parties and blocs would suggest. This is the one graph with a clear gain left after removing free positives (+5.18), so multi-scale carries some real signal here.

UN Trade: nothing to add at the feature level, much to fix in training. Dense and persistent, with a long tail in its kernel. The tuned rate keeps 0.043 per step, so old contacts stay visible, and only 2.0% of positives are free. Direct contact alone reaches AUC 0.92 and single-rate features reach 92.5, so the ladder has no room (−0.79). Yet neural TPNet scores 61.00, because random training rewards familiarity and every historical negative is familiar. This is why MuST-HN gains the most here (+26.48 over TPNet). Direction matters too, with 23.2% of negatives active in reverse.

UN Vote: structure saturates. A near-complete graph, with density 0.981, transitivity 0.99 and degree Gini 0.01. Nearly every pair is active nearly every year, so walks carry almost no information about a specific pair. Common neighbours are 197 for positives and negatives alike (AUC 0.55). Effective contacts exceed d_R, so one contact sits at 0.81 noise units (Corollary 8). No positives are free. The only strong signal is direction, 96.27 aware against 55.05 blind, and by Proposition 17 the sketch cannot see it. Every model is therefore far below the rule. The +2.72 comes from one seed.

UCI: fast bursts in a sparse graph. Sparse and heavy-tailed, with Gini 0.65 and transitivity 0.06. Half of active nodes are gone a month later. Pairs return after a median 20 minutes, while the tuned half-life is 80 days, so the tuned rate is far too slow for this graph's clock. Feature-level AP is 56.57 at the tuned rate and 89.60 at 10^{-5}. But that rate is not in the ladder, because the ladder was built around $\lambda^\star$. The neural model recovers recency through its neighbour and time encoders, so more scales add no information it lacks (−1.02). Novelty is already visible at the tuned rate, and a no-learning rule reaches 97.36.

Enron: the tuned rate is right. A small, hub-driven graph with 56.5 edges per pair and a smooth kernel. The tuned half-life of 8 days matches a graph with a 90th-percentile gap of 21 days, and the best single rate equals the tuned rate. There is no headroom (+0.53 at feature level), so extra inputs cost a little (−2.35).

7.6 Method by method

MuST-Walk helps when two conditions hold together: many positives are novel, and the tuned rate is fast enough to tie them with idle pairs. US Legis. and Can. Parl. meet both. UN Trade and UN Vote lack novelty, UCI and Enron do not tie. This two-factor account fits five of six datasets; the UN Vote gain is the exception and rests on one seed. Width is not the limit on these graphs: in the feature-level test the projection costs 0.0 to 6.6 AP on the ladder (Table 15).

Ladder, historical AP	US Legis.	Can. Parl.	UN Trade	UN Vote	UCI	Enron
Exact walk features	90.50	89.94	92.79	64.37	89.11	90.46
Through a sketch of width d_R	90.51	86.27	89.32	63.09	82.54	88.81

Table 15: Cost of the projection at the width in use, feature level, first negative sample, three random projections.

MuST-HN gains on all six datasets against TPNet, 18 of 18 seeds. It gives the model the right sign: novelty means positive and familiarity alone means nothing. The effect appears in linear features too, where historical training lifts the ladder by $+4.27$ to $+22.11$ (Table 14). So the gain reflects the test's structure as well as better learning, and on four datasets a no-learning rule beats it. It costs random AP on 4 of 6 datasets. Since training and selection changed together we cannot split the gain between them. TGB, whose negatives are fixed, shows mixed results.

Gate and average have the largest gains over TPNet on US Legis. and were not tested elsewhere on corrected code, so we make no general claim for or against them.

8 Limitations

- Three seeds per configuration, and a per-dataset rule that noise can pass, so single-dataset gains are weak evidence. The cross-dataset test could not reach significance by design.
- Six DyGLib datasets are too few to establish a structural law. Our two-factor account is an explanation tested at feature level, not a proof.
- The feature-level analysis is linear and idealised. It uses edges that the neural pipeline holds out, and its test replicates the sampler rather than reusing it. It cannot show how the neural model decides.
- We did not run a neural single-rate sweep on corrected code, so we cannot say several rates beat one better rate in the neural model. At feature level, the ladder differs from the best single rate by $+3.47$, $+10.61$, -0.29, $+3.21$, -0.93 and $+0.53$.
- Training and selection changed together in MuST-HN, and it was not a pre-registered comparison against TPNet.
- Corrected-code ablations ran on US Legis. only.
- Models read the sketch at τ, not t (Proposition 13). Reading at t changes the tuned-rate feature-level result by -2.76 to $+1.84$ on yearly data, but we did not train on it.
- Tuned rates come from a pipeline with the leak, and the ladder is built around them. UCI shows the cost: its best rate is not in its ladder.
- Sketch width is below the bound of Theorem 10, so the projection has no worst-case guarantee at this width.
- The leak is isolated on one seed of one dataset.
- On TGB, strict time lowers seed 0 by more than the tied weight on DyGLib Enron and UCI suggests (0.636 to 0.5905, 0.283 to 0.1784). The cause is not established.
- tgbl-review received 5–9 epochs, unevenly across models, and tgbl-wiki and tgbl-review were not inspected at the pair level.
- Held-out-node edges are missing from the sketch during training, and hardware differs between some runs.

Whole experiment	Runs	Compute hours	Hit time limit
Main experiment	126	276.8	4
Large datasets	52	211.1	13
Ablations and other runs	54	46.8	0
Total	**232**	**534.7**	**17**

Table 16: Includes configurations outside this paper's scope. Sum of per-run durations from run logs, completed in about 45 elapsed hours (30 Oct to 1 Nov 2026) across Kaggle T4s, an RTX 6000 Ada and a laptop RTX 3050. Failed or restarted sessions and earlier original-code work are not counted.

9 Conclusion and future work

Several decay rates on one projection are cheap and mathematically clean. The sketch extends to J scales with no new approximation, cost grows linearly in J, and the lazy update removes the batch-wide write. They are not a general improvement. Their gains appear where a fast tuned rate ties never-seen pairs with long-idle ones, and most of that gain comes from the construction of the historical test, in which a never-seen pair cannot be negative. Graph structure decides the rest. Turnover and new pairs create the free positives, near-complete graphs saturate walks and exceed the sketch's resolution, direction is invisible to a symmetric sketch, and bursty streams need a much faster clock than the tuned rate provides.

Four directions follow. A historical test that does not give positives away, by also sampling unseen pairs as negatives, would measure forecasting more fairly. A neural single-rate sweep on corrected code would settle RQ1 directly. Ladders should be set from a graph's own kernel and timescale rather than around a tuned rate. A direction-aware update would address what the sketch cannot see. Until then, simple rules, including the novelty-aware one, should be reported beside any model on these benchmarks.

Reproducibility. Code, tests, pre-registrations, per-run results and the analysis script are at `https://github.com/MDP2004/MuST-Walk`.

References

[1] X. Lu, L. Sun, T. Zhu, W. Lv. Improving temporal link prediction via temporal walk matrix projection. *NeurIPS*, 2024.

[2] S. Huang et al. Temporal Graph Benchmark for machine learning on temporal graphs. *NeurIPS Datasets and Benchmarks*, 2023.

[3] F. Poursafaei et al. Towards better evaluation for dynamic link prediction. *NeurIPS Datasets and Benchmarks*, 2022.

[4] L. Yu, L. Sun, B. Du, W. Lv. Towards better dynamic graph learning: new architecture and unified library. *NeurIPS*, 2023.

[5] E. Rossi et al. Temporal graph networks for deep learning on dynamic graphs. *ICML GRL Workshop*, 2020.

[6] Y. Wang et al. Inductive representation learning in temporal networks via causal anonymous walks. *ICLR*, 2021.

[7] Y. Karmim et al. Supra-Laplacian encoding for transformer on dynamic graphs. *NeurIPS*, 2024.

[8] G. Cormode, V. Shkapenyuk, D. Srivastava, B. Xu. Forward decay: a practical time decay model for streaming systems. *ICDE*, 2009.

[9] J. Zhang, Z. Liu, Y. Ding, J. Wang, Y. Du, W. Xuan. Who is active: a missing factor that transfers across models. arXiv:2610.04869, 2026.

[10] M. Lampert, C. Blöcker, I. Scholtes. From link prediction to forecasting: addressing challenges in batch-based temporal graph learning. *TMLR*, 2026.

[11] Gao et al. Towards a better negative sampling strategy for dynamic graphs. *Neural Networks* 173, 2024.

[12] Chen, Zheng, Song. Curriculum negative mining for temporal networks. *Neural Networks* 191, 2025.

[13] Jiang, Zhang, Gao, Zheng. History doesn't repeat, but its patterns echo: a parallel pairwise negative-sampling framework for temporal link prediction. *IJCAI*, 2026.

[14] F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1945.

[15] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 1979.

[16] L. Isserlis. On a formula for the product-moment coefficient of any order of a normal frequency distribution in any number of variables. *Biometrika*, 1918.

[17] W. Johnson, J. Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 1984.